

Beyond Classical Search

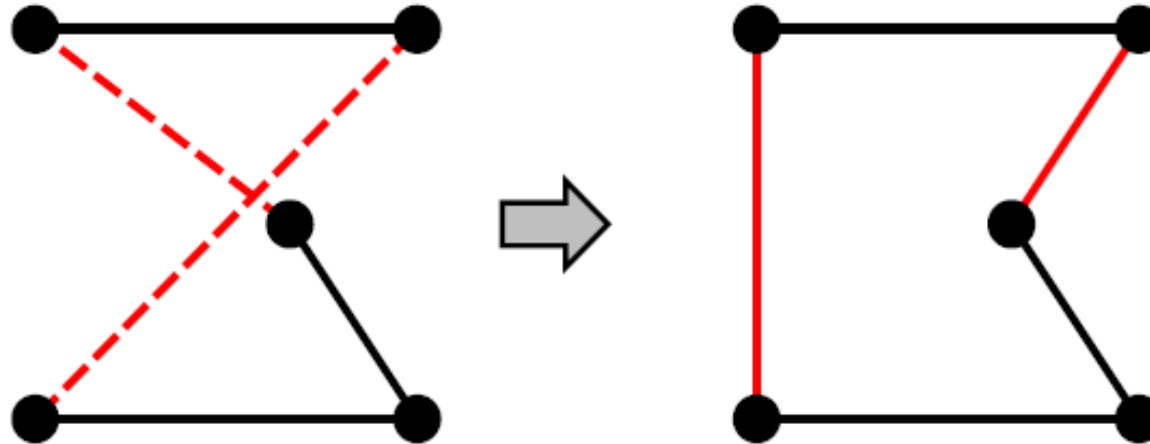
CHAPTER 4

Local Search – Iterative Improvement Algorithms

- ❑ For many problems, the solution path is irrelevant,
 - The goal state is itself the solution
- ❑ State space = set of “complete” configurations
 - Goal is to find an “optimal” configuration
- ❑ Iterative improvement algorithms
 - Keep a single “current” state
 - Try to improve it
- ❑ Constant space.

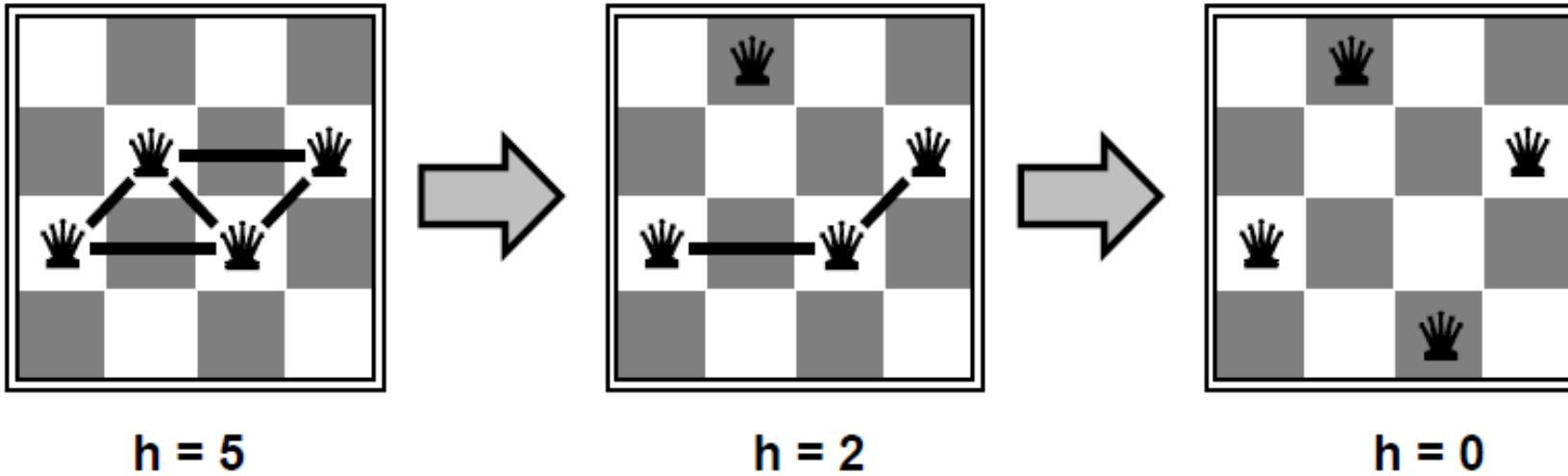
Example: TSP

- Start with any complete tour, perform pairwise exchanges



Example: n-Queens

- Move a queen to reduce number of conflicts



Local Search

- ❑ Hill Climbing
- ❑ Simulated Annealing
- ❑ Local Beam Search
- ❑ Genetic Algorithms

Hill Climbing

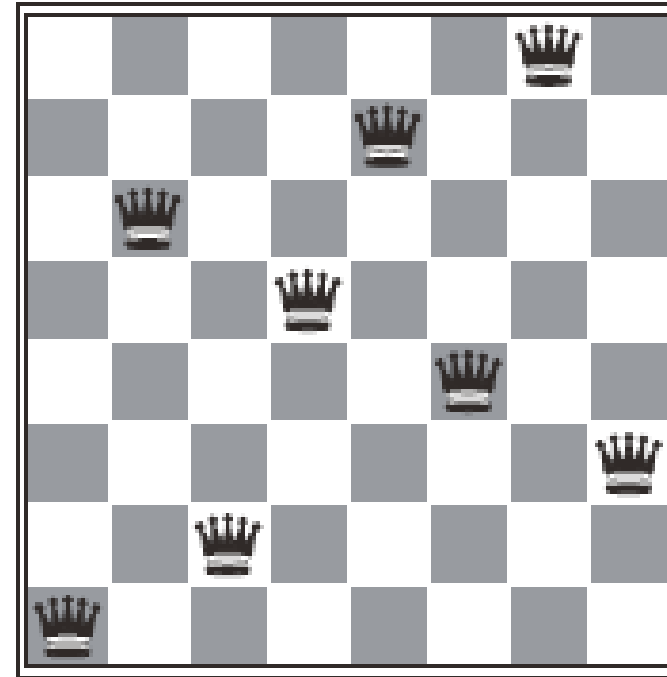
- ❑ Also referred to as steepest ascent
 - Greedy local algorithm
- ❑ Like climbing Mt. Everest in thick fog with amnesia

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                  neighbor, a node

  current ← MAKE-NODE(INITIAL-STATE[problem])
  loop do
    neighbor ← a highest-valued successor of current
    if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
    current ← neighbor
  end
```

Hill Climbing Example (1)

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♙	13	16	13	16
♙	14	17	15	♙	14	16	16
17	♙	16	18	15	♙	15	♙
18	14	♙	15	15	14	♙	16
14	14	13	17	12	14	12	18

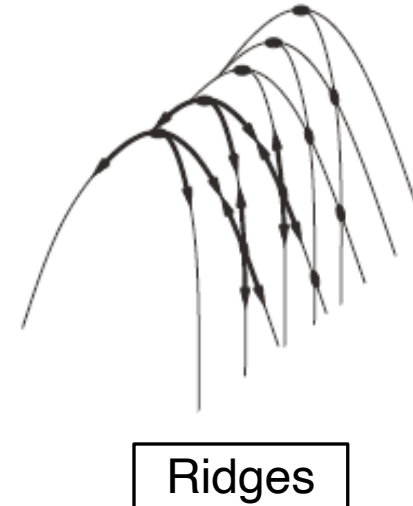
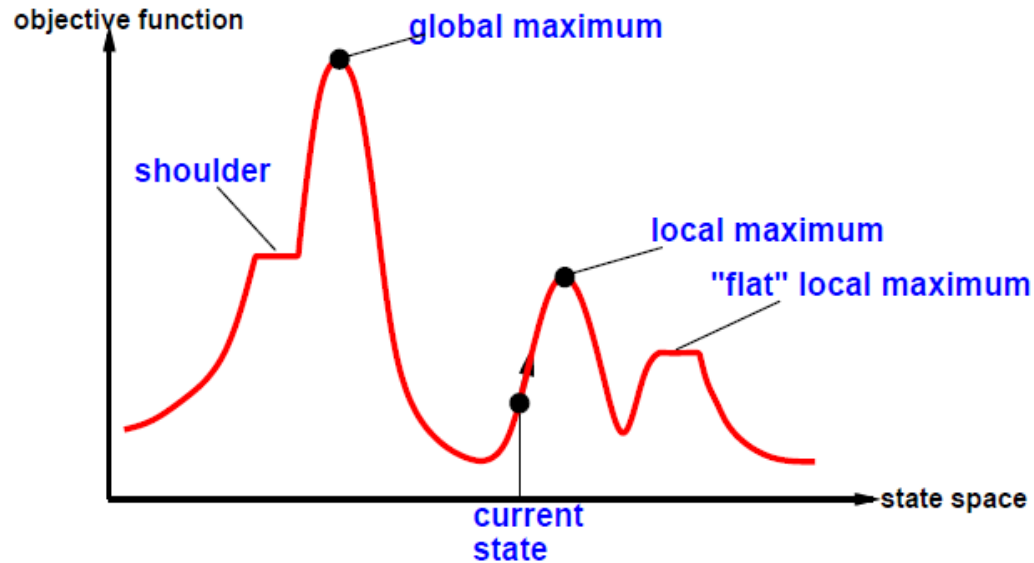


Hill Climbing Example (2)

- ❑ Randomly generated 8-queens starting states
- ❑ 14% of the time it solves the problem
- ❑ 86% of the time it gets stuck at a local minimum
- ❑ However
 - Takes only 4 steps on average when it succeeds
 - And 3 on average when it gets stuck
 - State-space size – $8^8 \sim 17M$ states

Hill Climbing

- ❑ Consider the state space landscape



- ❑ Random sideways moves
- ❑ Random-restart hill climbing
- ❑ Stochastic Hill Climbing

Random Sideways Moves

❑ If no downhill moves, allow sideways moves in hope that algorithm can escape

- Need to place a limit on the possible number of sideways moves

❑ For 8-queens

- Allow sideways moves with a limit of 100
- Raises percentage of problem instances solved from 14-94%
 - However, 21 steps for every successful solution and 64 for each failure.

❑ Tabu Search

- Keep track of states already visited

Variants of Hill Climbing

❑ Stochastic hill climbing

- Random selection from the available uphill moves

❑ First-choice hill climbing

- Pick the first successor that is better than the current state

❑ Random-restart hill climbing

- Hill climbing searches from randomly generated initial states

Simulated Annealing

- ❑ Idea: Escape local maxima by allowing some “bad” moves, but gradually decrease their size and frequency.
 - Hill climbing is efficient, but incomplete
 - Random walk is complete but inefficient
- ❑ Inspired by metallurgical process of ‘annealing’
 - The required low energy state is attained by first heating the material and then gradually cooling it
- ❑ Temperature Schedule – mapping time to temperature.
 - When temperature is high – allow random moves
 - When temperature is low – reduce the probability of random moves - More like hill climbing

Simulated Annealing

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to "temperature"
  local variables: current, a node
                    next, a node
                    T, a "temperature" controlling prob. of downward steps

  current ← MAKE-NODE(INITIAL-STATE[problem])
  for t ← 1 to ∞ do
    T ← schedule[t]
    if T = 0 then return current
    next ← a randomly selected successor of current
     $\Delta E \leftarrow \text{VALUE}[\textit{next}] - \text{VALUE}[\textit{current}]$ 
    if  $\Delta E > 0$  then current ← next
    else current ← next only with probability  $e^{\Delta E/T}$ 
```

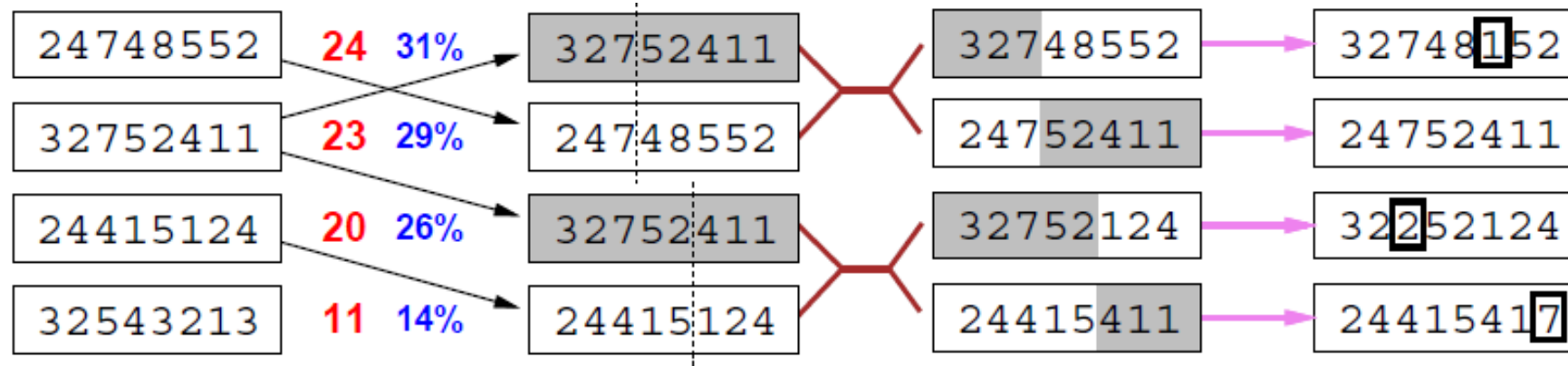
Local Beam Search

- ❑ Idea: Keep k states instead of 1; choose top k of all their successors
- ❑ Not same as searches run in parallel
 - Searches that find good states recruit other searches to join them
- ❑ Problem: all states end up on the same hill
- ❑ Modification: Stochastic local beam search
 - Pick successors randomly with bias towards good ones

Genetic Algorithms

- ❑ Idea: Combine stochastic local beam search and generating successors from pairs of states
- ❑ A state is represented as a string over a finite alphabet
 - 8-queens – position of the queens in each column
- ❑ Population - Start with k randomly generated states
- ❑ Evaluation Function (fitness function)
 - Higher values for better states
 - Opposite to heuristic functions
- ❑ Produce the next generation of states by “simulated evolution”
 - Random selection
 - Crossover
 - Random mutation

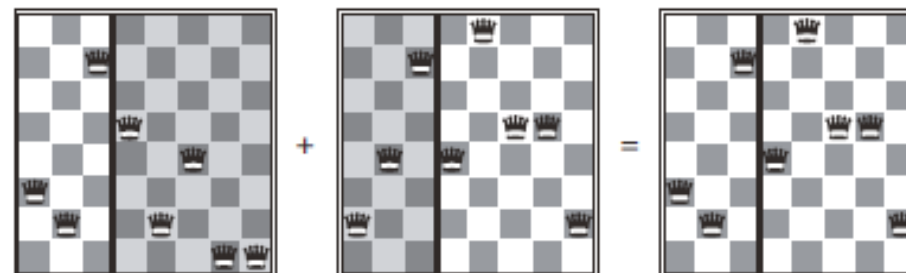
Genetic Algorithms



Fitness **Selection** **Pairs** **Cross-Over** **Mutation**

4 starting states

Fitness Function - Number of non-attacking pairs of queens
Probability of selection - proportional to the Fitness function



General Comments on GA

□ Pros

- Random exploration via crossovers can find solutions that local search cannot
- Connection to theory of human evolution

□ Cons

- Large number of tunable parameters
 - Difficult to replicate the performance from one problem to another (too much of randomization)
- Lack of good empirical studies comparing to other methods
- Useful on some problems, but no convincing evidence of its superiority over hill-climbing with random restarts.

Searching with Partial Observations

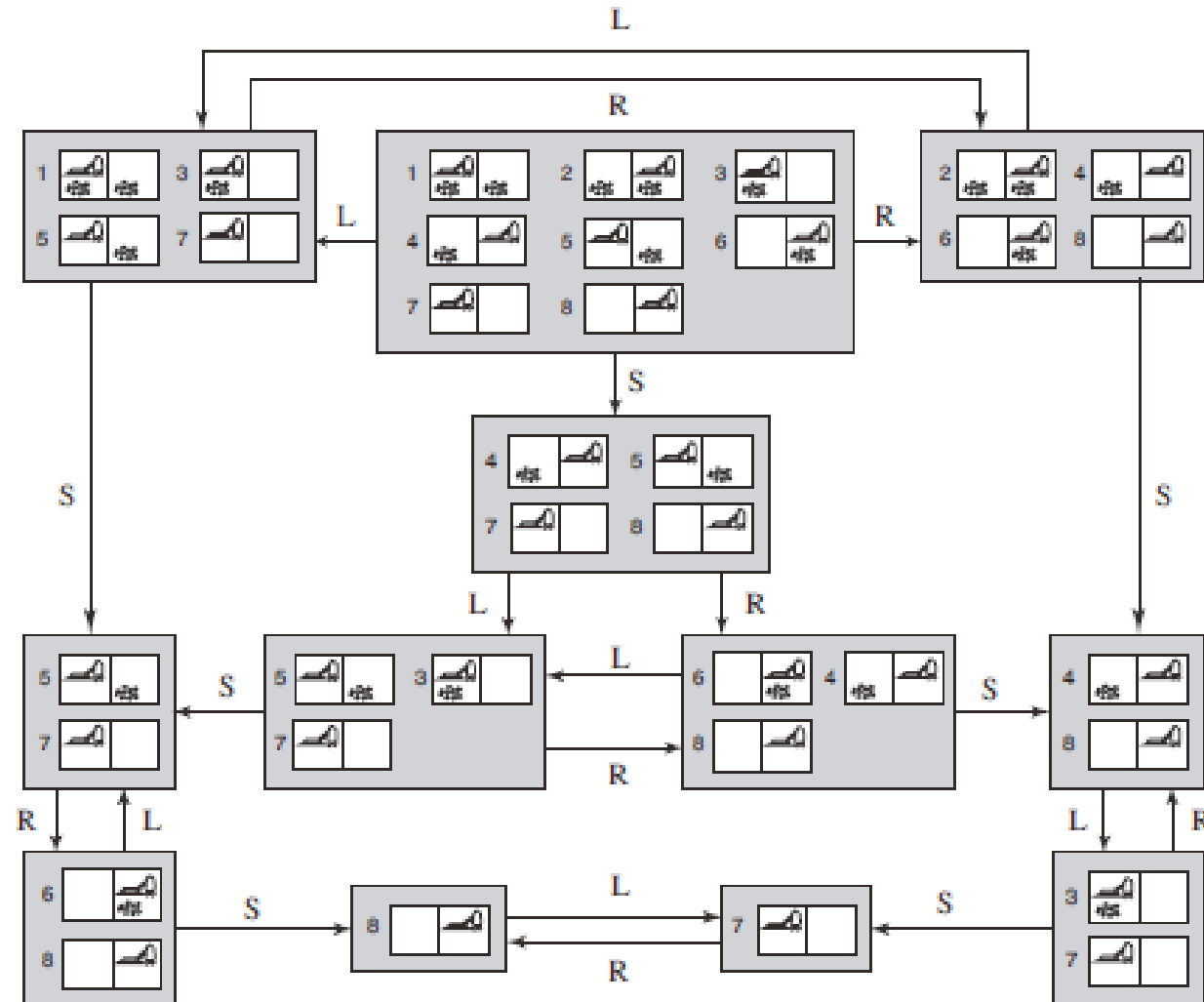
❑ State becomes belief state

- Agent's current belief about the possible physical states it might be in, given the sequence of actions and percepts up to that point

❑ Searching with no observation – conformant problem

- Doctors giving a prescription without a blood test

Vacuum world example



Searching with No Observation

□ If there are N states for a problem, then

- Belief state - $b \in$ power set of N states
- Initial state – typically a set of all the states in the problem
- Actions – Depends on the actions that are legal
 - Union of all actions allowed from every state in the belief state
 - Intersection of all actions allowed from every state in the belief state, if some of the actions are illegal
- Transition model
 - Actions are deterministic
 - Actions are nondeterministic
- Goal test – a belief state satisfies the goal test only if all the states in it satisfy the goal test